

Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

a) Write a C code to create a process using fork() system call.

b) Read the user input as shell command.

c) Use a system call exec() to execute the shell command.

d) Display the PIDs of parent and child processes.

```
Last login: Tue Aug  4 12:10:05 on ttys000
[vigneshkumar@vigneshs-MacBook-Air ~ % nano Task.c
vigneshkumar@vigneshs-MacBook-Air ~ %
```

UW PICO 5.09

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    char command[100];
    pid_t pid;

    // Read Linux command from user
    printf("Enter a Linux command: ");
    scanf("%99s", command);

    // Create child process
    pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }

    else if (pid == 0) {
        // Child process
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        // Execute command
        execlp(command, command, NULL);

        // Runs only if exec fails
        printf("Invalid command!\n");
        exit(1);
    }

    else {
        // Parent process
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        // Wait for child
        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}
```

2)Then press ctrl+oenter ctrl+x2)then gcc filename.c -o filename

```
[vigneshkumar@vigneshs-MacBook-Air ~ % nano Task.c
[vigneshkumar@vigneshs-MacBook-Air ~ % gcc Task.c -o Task
vigneshkumar@vigneshs-MacBook-Air ~ %
```

3)then ./filename

```
[vigneshkumar@vigneshs-MacBook-Air ~ % nano Task.c
[vigneshkumar@vigneshs-MacBook-Air ~ % gcc Task.c -o Task
[vigneshkumar@vigneshs-MacBook-Air ~ % ./Task
Enter a Linux command:
```

4)Enter the linux command

```
[vigneshkumar@vigneshs-MacBook-Air ~ % nano Task.c
[vigneshkumar@vigneshs-MacBook-Air ~ % gcc Task.c -o Task
[vigneshkumar@vigneshs-MacBook-Air ~ % ./Task
Enter a Linux command: ls

Parent Process
Parent PID: 10313
Child PID: 10316

Child Process
Child PID: 10316
Parent PID: 10313
a.out          Documents      first.c.save   Music
Applications   Downloads      Library        Pictures
Desktop         file1          Movies         Process
Child process completed.
vigneshkumar@vigneshs-MacBook-Air ~ %
```

Child process completed.

Task 2

Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

An operating system is system software that controls all the hardware and software resources of a computer. Users do not communicate directly with the hardware. Instead, the operating system acts as a bridge between the user, applications, and the hardware

Linux Commands Used

1. uname

The `uname` command is used to display information about the Linux operating system. It provides details such as the operating system name, kernel version, machine architecture, and hostname. This command helps us identify the system on which Linux is running.

```
[vigneshkumar@vigneshs-MacBook-Air ~ % uname  
Darwin  
vigneshkumar@vigneshs-MacBook-Air ~ %
```

2. lscpu

The `lscpu` command displays detailed information about the processor. It shows the CPU model, number of cores, number of threads, CPU architecture, and processor speed. This information helps us understand the hardware capabilities of the computer.

```
[vigneshkumar@vigneshs-MacBook-Air ~ % ./Task  
Enter a Linux command: lscpu  
  
Parent Process  
Parent PID: 10357  
Child PID: 10358  
  
Child Process  
Child PID: 10358  
Parent PID: 10357  
Invalid command!  
Child process completed.  
vigneshkumar@vigneshs-MacBook-Air ~ %
```

3. ps

The `ps` command lists the processes that are currently running in the system. It displays the Process ID (PID), Parent Process ID (PPID), user name, and the command associated with each process. This command is useful for monitoring and managing processes.

```
[vigneshkumar@vigneshs-MacBook-Air ~ % ps  
  PID TTY          TIME CMD  
 10283 ttys001    0:00.04 -zsh  
vigneshkumar@vigneshs-MacBook-Air ~ %
```

4. top

The top command provides a real-time view of the system's performance. It continuously displays CPU usage, memory usage, running processes, and system load. It is commonly used to identify programs that consume a large amount of system resources.

PID	COMMAND	%CPU	TIME	#TH	#WQ	#PORT	MEM	PURG	CMPRS	PGRP	PPID	STATE	BOOSTS
0	kernel_task	6.5	54:39.64	569/8	0	0	96M	0B	0B	0	0	running	0[0]
606	WindowServer	5.2	75:06.96	21	5	3639+	727M+	45M	257M	606	1	sleeping	*0[1]
10370	top	4.8	00:03.02	1/1	0	30	7472K	0B	0B	10370	10283	running	*0[1]
1128	spotlightkno	1.9	02:55.16	4	2	97+	15M+	0B	9728K	1128	1	sleeping	0[5732]
10078	Google Chrom	1.7	00:16.35	21	3	281-	403M+	0B-	57M	10072	10072	sleeping	*0[22151+]
10072	Google Chrom	1.4	00:46.76	46	1	891	175M	0B	90M	10072	1	sleeping	*10926[16]
686	com.apple.Dr	1.2	10:56.54	8	6	1406	32M-	0B	3248K	686	1	sleeping	0[1]
670	mDNSResponde	1.0	07:14.86	3	1	125-	9025K-	0B	2128K	670	1	sleeping	*0[1]
10079	Google Chrom	0.9	00:11.31	17	1	134	45M	0B	24M	10072	10072	sleeping	*3016[149]
561	coreduetd	0.8	00:16.83	6	5	139+	8433K+	9008K	2336K	561	1	sleeping	*1+[698+]
1083	corespotligh	0.8	02:00.48	7	5	376+	34M+	0B	19M	1083	1	sleeping	*1011[1]
731	BTLEServer	0.8	02:08.16	2	1	49	3264K	0B	1568K	731	1	sleeping	*0[1]
10374	screencaptur	0.7	00:00.62	2	1	76	6592K	16K	0B	1338	1338	sleeping	*0[883+]
591	bluetoothd	0.5	04:14.00	11	5	419	11M	160K	4368K	591	1	sleeping	*0[1]
1268	ControlCente	0.4	01:35.58	8	3	703	67M+	0B	44M	1268	1	sleeping	*0[7760]
600	cfprefsd	0.4	00:42.19	3	2	923+	4144K+	0B	1488K	600	1	sleeping	*1[2138]
10227	Terminal	0.3	00:11.65	9	4	331	162M	42M	13M	10227	1	sleeping	*0[171]
10141	Google Chrom	0.3	00:08.79	30	1	385	208M+	0B	80M	10072	10072	sleeping	*3[1523]
1008	suggestd	0.2	00:21.09	7	6	552+	17M+	128K	7680K	1008	1	sleeping	*1+[919+]
690	com.apple.Ap	0.2	00:50.85	2	1	121	1488K	0B	528K	690	1	sleeping	0[1]
9283	WhatsApp	0.2	01:34.92	29	5	903	228M	4272K	87M	9283	1	sleeping	*40[548]
10220	Google Chrom	0.1	00:08.01	21	1	244	289M	0B	225M	10072	10072	sleeping	*126[312]
594	corebrightne	0.1	01:01.17	3	2	143	4688K	0B	2048K	594	1	sleeping	*0[1]
10066	netbiosd	0.1	00:01.92	7	6	30	3024K	0B	1072K	10066	1	sleeping	*0[1]
530	fseventsd	0.1	01:03.47	12	1	173	5040K	0B	2048K	530	1	sleeping	*0[1]
6637	mysqld	0.1	00:31.68	37	0	60	470M	0B	468M	6534	6534	sleeping	*0[1]
641	airportd	0.1	08:22.12	8	6	394	17M	0B	6976K	641	1	sleeping	*1[1]
679	audioclocksy	0.0	00:29.24	3	2	51	2816K	0B	1536K	679	1	sleeping	*0[1]
1140	useractivity	0.0	00:04.84	2	1	107	3856K	0B	1040K	1140	1	sleeping	*0[4455+]
526	logd	0.0	03:08.76	4	3	1851	23M	0B	23M	526	1	sleeping	*0[1]
581	distnoted	0.0	00:11.40	3	2	200	2224K	0B	752K	581	1	sleeping	*0[1]
555	mds	0.0	01:44.75	7	4	386	42M	0B	34M-	555	1	sleeping	*0[1]
1	launchd	0.0	03:17.10	3	2	4491+	25M+	0B	7760K	1	0	sleeping	0[0]
958	nsurlsession	0.0	00:23.08	9	6	145+	7696K+	0B	3952K	958	1	sleeping	0[4428]
995	sharingd	0.0	01:16.24	4	1	399	21M	0B	14M	995	1	sleeping	*0[1]
1339	Finder	0.0	02:22.28	8	2	641	173M	0B	155M	1339	1	sleeping	*0[3038]
565	contextstore	0.0	00:21.21	3	2	153	6624K	0B	3328K	565	1	sleeping	*0[29900+]
1040	fileprovider	0.0	00:28.62	3	2	154	26M	32K	21M	1040	1	sleeping	*164[36]
8857	PerfPowerSer	0.0	01:26.01	7	3	507	15M	320K	4864K	8857	1	sleeping	0[2187]
1242	duetexpertd	0.0	09:51.44	2	1	418	19M	1168K	5136K	1242	1	sleeping	0[1477]
1437	jetpackasset	0.0	00:01.85	3	2	80+	6512K+	0B	5168K	1437	1	sleeping	0[67]
966	lockoutagent	0.0	00:08.05	4	3	59	3168K	0B	1472K	966	1	sleeping	*0[1]
10373	AXVisualSupp	0.0	00:00.07	3	1	150+	9121K+	0B	0B	10373	1	sleeping	*0[5]